

PROPOSAL PENELITIAN

**PERANCANGAN FRAMEWORK LAPISAN
BACKEND FULLY HOMOMORPHIC ENCRYPTION
DAN STANDAR PENGEMBANGAN APLIKASI
BERBASIS PRIVASI DATA**



Disusun Oleh:

Muhammad Ulinnuha

23/516588/PA/22101

**DEPARTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN
ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

Daftar Isi

I	PENDAHULUAN	1
1.1	Latar Belakang	1
1.2	Rumusan Masalah	3
1.3	Batasan Masalah	4
1.4	Tujuan Penelitian	4
1.5	Sistematika Penulisan	5
II	TINJAUAN PUSTAKA	6
2.1	Permasalahan Penyediaan FHE bagi Pengembang Aplikasi	6
2.2	Solusi yang Telah Ada	7
2.3	Gap dan Posisi Penelitian	8
2.4	Landasan Teori	9
III	METODOLOGI PENELITIAN	11
3.1	Tahapan Penelitian	11
3.2	Stack dan Artefak Prototipe	11
3.3	Rancangan Framework Backend	12
3.4	Standar Interaksi Aplikasi dan Backend	13
3.5	Skenario Pengujian	13
3.6	Metode Analisis	14
IV	RENCANA PENELITIAN	15
4.1	Prototipe HEBA	15
4.2	Rencana Implementasi dan Status	15
4.3	Komponen dan Alur Kerja Prototipe	17
4.4	Rencana Hasil dan Pembahasan	17
4.5	Keterbatasan dan Arah Pengembangan	18
A	STANDAR INTERAKSI APLIKASI DAN BACKEND	21

BAB I

PENDAHULUAN

1.1 Latar Belakang

Komputasi di lingkungan *cloud* dan layanan pihak ketiga telah menjadi fondasi banyak sistem informasi modern. Fleksibilitas, skalabilitas, dan efisiensi biaya mendorong organisasi menempatkan penyimpanan serta pemrosesan data di luar infrastruktur miliknya sendiri [1]. Pada saat yang sama, data yang diproses sering mengandung informasi sensitif. Di bidang kesehatan, misalnya, rekam medis dan hasil analisis klinis menuntut perlindungan kerahasiaan yang ketat [2]. Di skenario multi-pihak, nilai pengukuran atau metrik kinerja dapat membocorkan keunggulan metode atau data klien apabila dibuka dalam bentuk plaintext [3]. Enkripsi konvensional melindungi data saat disimpan atau dikirim, tetapi biasanya mewajibkan dekripsi sebelum komputasi. Begitu plaintext muncul di server pemroses, risiko akses tidak sah dan kebocoran data kembali terbuka [1, 3].

Fully Homomorphic Encryption (FHE) menawarkan jalur lain. Data dienkripsi di sisi pemilik, dikirim ke server, lalu diolah tetap dalam bentuk ciphertext. Hasil evaluasi juga terenkripsi dan baru dibuka oleh pihak yang memegang kunci privat [4, 2]. Alur tersebut membuat FHE cocok untuk komputasi di server yang tidak sepenuhnya dipercaya, termasuk layanan *cloud* dan analisis bersama antar lembaga. Dalam literatur, FHE sudah dihubungkan dengan privasi di layanan kesehatan, komputasi statistik rahasia, dan alur kerja yang dirancang agar kerahasiaan dijaga sejak perancangan sistem [2, 5, 3].

Persoalan yang diangkat di sini bukan lagi soal apakah FHE mampu menghitung tanpa dekripsi. Kemampuan itu sudah menjadi inti skema FHE. Persoalan yang lebih mendesak bagi rekayasa perangkat lunak adalah bagaimana menempatkan FHE di dalam sistem sehingga pengembang aplikasi tidak terpaksa membangun seluruh infrastruktur kriptografi setiap kali membuat layanan baru. Pada praktiknya, pengembang yang ingin memakai FHE sering harus memilih skema, mengatur parameter keamanan, memahami pertumbuhan *noise*, memperhitungkan kedalaman operasi, mengelola kunci, menyiapkan format ciphertext, dan mengukur biaya evaluasi. Studi kelayakan komputasi statistik dengan TFHE melalui compiler Concrete memperlihatkan betapa cepatnya kompleksitas sirkuit, jumlah *programmable bootstrapping*,

waktu kompilasi, dan waktu eksekusi membengkak bahkan untuk ukuran data yang masih kecil [5]. Survei optimasi *bootstrapping* menegaskan bahwa penyegaran *noise* dapat menghabiskan sebagian besar waktu eksekusi FHE [6]. Analisis di GPU menunjukkan bahwa ukuran ciphertext membengkak dan latensi evaluasi masih jauh di atas komputasi plaintext [4]. Beban-beban itu bersifat sistemik. Apabila setiap tim aplikasi harus menyelesaikannya sendiri, pengembangan layanan berbasis privasi menjadi mahal, sulit direplikasi, dan rawan inkonsistensi.

Kebutuhan akan pemisahan tanggung jawab antara pengembang aplikasi dan penyedia infrastruktur sebenarnya sudah familiar di model layanan *cloud*. Pada model *Platform as a Service* (PaaS), pengembang dapat memusatkan perhatian pada pembuatan dan penerapan aplikasi di lingkungan yang sudah distandarkan, tanpa harus mengelola seluruh administrasi infrastruktur dasar [1]. Logika yang sama relevan untuk FHE. Pengembang aplikasi semestinya dapat menulis logika bisnis dan mematuhi kontrak interaksi yang jelas, sementara kerangka sistem di lapisan backend menyiapkan konfigurasi skema, layanan evaluasi homomorfik, pengelolaan kunci sesuai peran, serta batasan operasi yang diizinkan.

Literatur yang ada telah menyiapkan fondasi di beberapa arah. Skema CKKS banyak dibahas untuk komputasi numerik aproksimasi, termasuk integrasinya dengan pembelajaran mesin [7]. TFHE dan compiler yang menyertainya memberi jalur lain untuk operasi Boolean dan bilangan bulat [5]. Penelitian arsitektur perangkat lunak untuk proteksi data di *cloud* dengan enkripsi homomorfik menguraikan modul klien, penyimpanan atau pemroses di *cloud*, manajemen kunci, algoritma enkripsi dan evaluasi, serta infrastruktur jaringan, lengkap dengan alur pengiriman ciphertext, permintaan operasi, dan pengembalian hasil terenkripsi [1]. Pendekatan *secure-by-design* pada pengujian laboratorium menunjukkan bahwa pembagian peran antara pihak yang memiliki data, pihak yang mengevaluasi, dan pihak yang memegang kunci dapat disusun menjadi alur kerja yang tegas [3]. Di sisi performa, riset akselerasi GPU, FPGA, dan arsitektur kluster evaluasi homomorfik berusaha menurunkan hambatan rotasi, bandwidth memori, dan bottleneck mikroarsitektur agar FHE lebih layak dijalankan di server [4, 8, 9].

Meskipun demikian, pembahasan tersebut sering berhenti pada skema kriptografi, kasus aplikasi tertentu, atau akselerasi komputasi. Yang masih kurang adalah kerangka backend generik beserta standar yang mengatur bagaimana aplikasi boleh berinteraksi dengan layanan FHE. Tanpa kerangka dan

standar itu, setiap proyek cenderung merakit ulang alur kriptografi, aturan kunci, dan format permintaan dengan caranya sendiri. Pengembang aplikasi pun tetap terdorong masuk ke detail yang semestinya menjadi urusan lapisan sistem.

Penelitian ini mengusulkan perancangan framework lapisan backend FHE beserta standar pengembangan aplikasi yang menyertainya. Framework yang dimaksud adalah rancangan arsitektur layanan di sisi sistem: konfigurasi parameter, layanan evaluasi pada ciphertext, pengelolaan kunci menurut peran, antarmuka layanan, dan pencatatan metrik. Standar yang dimaksud adalah aturan yang wajib diikuti aplikasi ketika memakai backend tersebut, meliputi format permintaan, batasan operasi, larangan mengirim kunci privat ke server evaluasi, dan kewajiban menerima hasil tetap dalam bentuk terenkripsi. Dengan pemisahan ini, pengembang aplikasi diharapkan dapat fokus pada logika aplikasi, sementara kompleksitas FHE dipusatkan di kerangka backend.

Sebagai wujud yang dapat diuji, penelitian membangun prototipe bernama **HEBA** (*Homomorphic Encryption Backend Architecture*). HEBA adalah kerangka backend evaluasi FHE beserta standar interaksi bagi pengembang aplikasi. Stack yang dipakai pada implementasi awal adalah skema CKKS melalui TenSEAL di balik antarmuka REST FastAPI, dengan jalur pengembangan adapter OpenFHE di kemudian hari [10, 11, 12]. Klien Python tipis hanya mengenkripsi masukan, memanggil API, dan mendekripsi keluaran. Beban kerja utama adalah skoring linear terenkripsi $y = \mathbf{w}^\top \mathbf{x} + b$ pada vektor fitur berdimensi tetap, dilengkapi penjumlahan dan perkalian skalar sebagai unit verifikasi alur.

1.2 Rumusan Masalah

Berdasarkan latar belakang tersebut, penelitian ini merumuskan masalah tentang bagaimana merancang framework lapisan backend Fully Homomorphic Encryption beserta standar interaksi yang mengatur hubungan antara aplikasi dan backend, sehingga pengembang dapat membangun aplikasi berbasis privasi data tanpa harus menyusun dan mengoperasikan seluruh infrastruktur FHE dari awal. Rumusan tersebut mencakup kebutuhan akan arsitektur layanan yang menempatkan evaluasi homomorfik di sisi sistem, serta kebutuhan akan aturan kepatuhan yang membuat perilaku aplikasi tetap selaras dengan batas keamanan dan kelayakan FHE.

1.3 Batasan Masalah

Penelitian ini dibatasi pada perancangan framework lapisan backend FHE, penyusunan standar interaksi aplikasi–backend, dan implementasi prototipe layanan evaluasi yang dapat dijalankan. Fokus penelitian tidak diarahkan untuk menghasilkan skema kriptografi baru ataupun membuktikan keamanan matematis baru. Akselerasi perangkat keras, komputasi terdistribusi, dan optimasi *bootstrapping* dibahas sebagai konteks beban di lapisan sistem, tetapi bukan objek rekayasa utama. Standar yang disusun bersifat kerangka akademik: kontrak data, pembagian peran, batasan operasi, dan alur validasi. Standar tersebut tidak digarap sebagai spesifikasi industri formal.

Stack implementasi dipatok pada OpenFHE dengan skema CKKS, layanan HTTP REST menggunakan FastAPI, dan klien contoh dalam Python. Operasi yang didukung backend dibatasi pada himpunan tetap: penjumlahan dua ciphertext, perkalian ciphertext dengan vektor plaintext (bobot), dan evaluasi skoring linear $y = \mathbf{w}^\top \mathbf{x} + b$ untuk vektor fitur berdimensi $d \in \{8, 16\}$. Bootstrapping tidak dijalankan pada prototipe; kedalaman multiplikasi disesuaikan agar skoring linear dapat dievaluasi dalam satu putaran tanpa penyegaran *noise*. Evaluasi mencatat kepatuhan terhadap standar, galat relatif terhadap baseline plaintext, ukuran ciphertext, serta waktu enkripsi, evaluasi, dan dekripsi.

1.4 Tujuan Penelitian

Penelitian ini bertujuan merancang framework lapisan backend FHE yang menempatkan konfigurasi skema, evaluasi homomorfik, pengelolaan kunci menurut peran, dan antarmuka layanan di dalam kerangka sistem. Penelitian ini juga bertujuan merumuskan standar pengembangan aplikasi yang mengatur cara aplikasi berinteraksi dengan backend tersebut, mencakup format permintaan REST, batasan operasi, aturan penanganan kunci, dan kewajiban menjaga data tetap terenkripsi selama berada di server evaluasi. Tujuan implementatifnya adalah membangun prototipe HEBA (backend CKKS berbasis FastAPI, klien tipis, dan penegakan standar), menjalankan skenario skoring linear terenkripsi, lalu mengukur kebenaran hasil serta biaya dasar eksekusi.

1.5 Sistematika Penulisan

Dalam penulisan laporan ini, susunan pembahasan mengikuti sistematika berikut:

BAB I : PENDAHULUAN

Bagian awal laporan memaparkan alasan penelitian dilakukan, masalah yang ingin dijawab, batasan pembahasan, tujuan yang hendak dicapai, manfaat penelitian, dan susunan penulisan.

BAB II : TINJAUAN PUSTAKA

Pembahasan diarahkan pada permasalahan penyediaan FHE bagi pengembang aplikasi, solusi yang telah ada, gap penelitian, landasan teori yang relevan, dan posisi rancangan backend yang diusulkan.

BAB III : METODOLOGI PENELITIAN

Pada bagian ini dijelaskan tahapan penelitian, rancangan framework backend, perumusan standar interaksi aplikasi, skenario pengujian, metrik evaluasi, dan metode analisis.

BAB IV : RENCANA PENELITIAN

Bagian akhir proposal menjelaskan prototipe HEBA, diagram arsitektur dan alur evaluasi, status implementasi, bentuk hasil, keterbatasan, serta arah pengembangan. Lampiran A merangkum standar interaksi aplikasi-backend.

BAB II

TINJAUAN PUSTAKA

2.1 Permasalahan Penyediaan FHE bagi Pengembang Aplikasi

FHE memungkinkan komputasi pada data terenkripsi, tetapi kemampuan itu tidak otomatis membuat FHE mudah dipakai di dalam produk perangkat lunak. Pada aplikasi biasa, pengembang menulis fungsi, mengirim data ke server, memprosesnya, lalu mengembalikan hasil. Pada FHE, data tidak boleh dibuka selama berada di lingkungan evaluasi. Setiap keputusan tentang tipe data, kedalaman operasi, dan bentuk fungsi memengaruhi apakah komputasi masih dapat dijalankan secara homomorfik [3, 7]. Ciphertext membawa parameter, *noise*, dan batasan operasi. Fungsi yang tampak ringan pada plaintext dapat menjadi mahal atau tidak layak setelah diterjemahkan ke sirkuit FHE [5].

Beban teknis tersebut muncul di banyak lapisan sekaligus. Pemilihan skema menentukan jenis aritmetika yang didukung. CKKS cocok untuk komputasi numerik aproksimasi, sementara TFHE lebih dekat dengan operasi Boolean dan bilangan bulat [7, 5]. Pertumbuhan *noise* membatasi kedalaman evaluasi; *bootstrapping* diperlukan untuk menyegarkan ciphertext, tetapi proses itu sering menjadi sumber biaya dominan [6]. Ukuran ciphertext juga membengkak dibandingkan plaintext, sehingga bandwidth, memori, dan latensi menjadi perhatian serius pada server evaluasi [4]. Ketika pengembang aplikasi harus mengurus seluruh rantai itu, pekerjaan membangun fitur bisnis tumpang tindih dengan pekerjaan membangun infrastruktur kriptografi.

Di lingkungan *cloud*, risiko keamanan memperparah kebutuhan akan desain sistem yang tegas. Ancaman kebocoran data, akses tidak sah melalui antarmuka yang lemah, dan manipulasi selama penyimpanan atau pemrosesan menuntut agar data sensitif tidak diolah dalam bentuk plaintext di server pihak ketiga [1]. FHE menjawab kebutuhan itu secara kriptografis, tetapi tanpa kerangka backend yang terstandar, setiap tim aplikasi cenderung merancang ulang alur kunci, format payload, dan aturan evaluasi. Hasilnya adalah implementasi yang sulit dibandingkan, sulit diaudit, dan sulit diwariskan ke pengembang lain.

2.2 Solusi yang Telah Ada

Pada lapisan skema dan perangkat pengembang kriptografi, CKKS banyak dikaji untuk komputasi aproksimasi pada bilangan real atau kompleks, termasuk skenario pembelajaran mesin yang masih dapat menerima galat kecil [7]. TFHE, melalui compiler seperti Concrete, memperlihatkan jalur dari program level tinggi ke komputasi homomorfik, sekaligus mengungkap batasan praktis terkait ukuran data, kompleksitas sirkuit, dan waktu eksekusi [5]. Kedua jalur itu memperkaya kemampuan FHE, tetapi masih meninggalkan pertanyaan tentang di mana detail skema seharusnya disembunyikan dari pengembang aplikasi dan di mana detail itu harus tetap dikendalikan oleh sistem.

Pada lapisan pustaka FHE, OpenFHE menyediakan implementasi skema-skema utama (termasuk CKKS) dalam satu API, dengan mode yang ramah pengguna maupun mode yang lebih terbuka bagi pengendali eksternal, serta abstraksi perangkat keras melalui *Hardware Abstraction Layer* [11]. Karakterisasi OpenFHE pada skema CKKS menunjukkan bahwa biaya dan perilaku performa sangat bergantung pada parameter dan tingkat abstraksi workload yang dijalankan, sehingga konfigurasi sepatutnya dikelola secara terpusat di sisi sistem, bukan diulang di setiap aplikasi [12]. Di ekosistem Python, pustaka seperti TenSEAL membungkus operasi tensor terenkripsi di atas mesin SEAL dan memudahkan eksperimen di level aplikasi [10]. Penelitian ini tidak membangun pustaka FHE baru; OpenFHE dipakai sebagai mesin evaluasi di backend, sementara kontribusi terletak pada kerangka layanan dan standar pemakaiannya.

Pada lapisan arsitektur perangkat lunak, penelitian tentang enkripsi homomorfik untuk proteksi data di *cloud* menguraikan tujuan dan persyaratan perangkat lunak, lalu memecah sistem menjadi aplikasi klien, penyimpanan atau pemroses di *cloud*, modul manajemen kunci, algoritma enkripsi dan pemrosesan data, serta infrastruktur jaringan [1]. Aplikasi klien mengirim data terenkripsi dan dapat meminta operasi aritmetika pada ciphertext. Komponen di *cloud* menerima ciphertext, menjalankan operasi tanpa dekripsi, dan mengembalikan hasil terenkripsi. Modul kunci menangani generasi, penyimpanan, dan distribusi kunci sesuai kebutuhan skema. Arsitektur itu menunjukkan bahwa FHE dapat diposisikan sebagai sistem layanan dengan batas tanggung jawab antar komponen, bukan sekadar pemanggilan pustaka di dalam kode aplikasi.

Pendekatan *secure-by-design* pada pengujian kompetensi laboratorium memberi contoh alur multi-pihak yang lebih ketat. Penyelenggara menyediakan

an nilai acuan terenkripsi dan material kunci yang relevan; peserta mengenkripsi data pengukuran di sisi lokal, mengevaluasi metrik pada ciphertext, lalu mengembalikan hanya metrik terenkripsi; dekripsi hasil akhir dilakukan tanpa mengharuskan penyelenggara menerima plaintext pengukuran mentah peserta [3]. Alur tersebut menegaskan pentingnya pemisahan peran dan larangan membuka data di titik yang tidak berwenang.

Pada lapisan performa, berbagai penelitian menempatkan bottleneck FHE di domain sistem dan perangkat keras. Survei *bootstrapping* memetakan optimasi untuk skema word-wise, bit-wise, dan hibrida, serta menekankan porsi waktu yang dihabiskan proses penyegaran *noise* [6]. Analisis FHE di GPU mengidentifikasi hambatan mikroarsitektur yang menurunkan utilisasi perangkat keras [4]. Akselerator FPGA dan arsitektur kluster evaluasi homomorfik menargetkan operasi mahal seperti rotasi dan transformasi terkait bootstrapping [8, 9]. Temuan-temuan itu memperkuat argumen bahwa urusan performa dan eksekusi evaluasi wajar digarap di lapisan backend sistem, bukan dibebankan berulang-ulang kepada setiap pengembang fitur aplikasi.

2.3 Gap dan Posisi Penelitian

Solusi yang tersedia sudah memperkuat fondasi FHE di ranah skema, kasus terapan, arsitektur *cloud*, dan akselerasi. Namun, kebutuhan pengembangan aplikasi yang ingin memakai FHE tanpa membangun infrastruktur dari nol masih belum terjawab secara utuh. Compiler membantu penerjemahan sebagian fungsi. Arsitektur *cloud* memberi contoh modul. Akselerator mempercepat potongan eksekusi. Yang masih jarang diformulasikan sebagai objek perancangan tersendiri adalah framework backend generik yang mengekspos layanan evaluasi, ditambah standar yang menjelaskan kewajiban aplikasi ketika memakai layanan itu.

Tanpa standar, antarmuka backend mudah digelincirkan menjadi sekumpulan pemanggilan teknis yang tetap memaksa pengembang memahami parameter FHE secara mendalam. Sebaliknya, tanpa backend yang disiplin, standar tertulis sulit ditegakkan karena tidak ada titik penegakan di sistem. Penelitian ini memandang keduanya sebagai satu paket rancangan. Framework backend menyediakan tempat konfigurasi, evaluasi, pengelolaan kunci menurut peran, dan pengukuran. Standar menyediakan aturan main bagi aplikasi: data masuk dan keluar dalam bentuk yang diizinkan, operasi dibatasi pada himpunan yang didukung parameter aktif, kunci privat tidak pernah diserahkan ke server evaluasi, dan hasil tetap terenkripsi hingga tiba di pihak

berwenang.

Kontribusi yang diusulkan bukan skema kriptografi baru dan bukan akselerator baru. Kontribusi berada pada perancangan kerangka sistem backend FHE dan standar pengembangan aplikasi yang menyertainya, agar pengembangan dapat memusatkan kerja pada logika aplikasi sementara infrastruktur FHE disiapkan dan dikendalikan di lapisan backend.

2.4 Landasan Teori

Fully Homomorphic Encryption adalah bentuk enkripsi yang memungkinkan operasi dijalankan langsung pada ciphertext dan menghasilkan ciphertext baru yang, setelah didekripsi oleh pemegang kunci yang sah, bersesuaian dengan hasil operasi pada plaintext [2, 1]. Berbeda dengan enkripsi biasa yang menuntut dekripsi sebelum data diolah, FHE menjaga data tetap tersembunyi selama pemrosesan di server. Sifat itu membuat FHE relevan untuk komputasi di infrastruktur yang tidak sepenuhnya dipercaya.

Dalam pelaksanaan, setiap operasi homomorfik menambah biaya komputasi dan biasanya menambah *noise* pada ciphertext. Apabila *noise* melampaui ambang tertentu, dekripsi gagal atau menghasilkan nilai yang salah. *Bootstrapping* menyegarkan ciphertext agar evaluasi dapat dilanjutkan, tetapi proses itu sering menjadi bagian paling mahal dalam eksekusi FHE [6]. Pemahaman terhadap kedalaman komputasi dan biaya penyegaran *noise* menjadi dasar mengapa konfigurasi parameter dan kebijakan operasi seharusnya dikelola di backend, bukan ditetapkan ad hoc di setiap aplikasi.

CKKS mendukung aritmetika aproksimasi pada data numerik dan banyak dipakai ketika galat kecil masih dapat diterima, misalnya pada sejumlah skenario analisis dan pembelajaran mesin [7, 3]. TFHE menempuh jalur berbeda dengan penekanan pada operasi Boolean dan bilangan bulat, serta *programmable bootstrapping* untuk fungsi yang lebih kaya pada level bit [5]. Pemilihan skema memengaruhi bentuk data yang dapat diproses dan jenis operasi yang wajar disediakan backend.

Framework lapisan backend dalam penelitian ini dipahami sebagai kerangka layanan sistem yang berdiri di antara aplikasi dan mesin kriptografi FHE. Backend menyiapkan parameter skema, mengekspos antarmuka permintaan evaluasi, menjalankan operasi pada ciphertext, mengelola material kunci sesuai peran, dan mencatat metrik layanan. Aplikasi di sisi klien tetap bertanggung jawab atas logika bisnis, enkripsi input di pihak yang berwenang, serta dekripsi hasil akhir, tetapi tidak menyusun ulang infrastruktur evaluasi.

Standar pengembangan aplikasi dipahami sebagai seperangkat aturan kepatuhan yang mengikat cara aplikasi memakai backend. Aturan itu mencakup format payload, metadata operasi yang wajib disertakan, batasan jenis operasi, larangan mengirim kunci privat ke server evaluasi, dan kewajiban menerima keluaran tetap terenkripsi. Standar berfungsi sebagai kontrak antara pengembang aplikasi dan penyedia kerangka backend.

Evaluasi rancangan tidak cukup melihat apakah program berjalan. Kebenaran alur diuji dengan membandingkan hasil dekripsi terhadap baseline plaintext, dengan toleransi yang wajar apabila skema bersifat aproksimasi [3]. Metrik performa dasar meliputi waktu enkripsi di sisi berwenang, waktu evaluasi di backend, waktu dekripsi, dan ukuran ciphertext. Metrik tersebut dibaca sebagai gambaran biaya layanan, sejalan dengan cara literatur memosisikan overhead FHE sebagai tantangan sistem [6, 4, 9].

BAB III

METODOLOGI PENELITIAN

3.1 Tahapan Penelitian

Penelitian berlangsung melalui tahapan yang berurutan. Tahap pertama adalah studi literatur tentang FHE, arsitektur proteksi data di *cloud*, alur multi-pihak, karakteristik OpenFHE/CKKS, serta tantangan performa evaluasi homomorfik. Tahap kedua adalah analisis kebutuhan framework backend dan standar interaksi, dengan memetakan tanggung jawab yang wajib berada di sistem (parameter, evaluasi, penegakan whitelist operasi) dan yang tetap di klien (enkripsi masukan, dekripsi hasil, logika bisnis). Tahap ketiga adalah perancangan arsitektur layanan dan kontrak REST. Tahap keempat adalah implementasi prototipe pada stack yang sudah ditetapkan. Tahap kelima adalah pengujian skoring linear terenkripsi beserta kasus penolakan standar, lalu analisis kebenaran, kepatuhan, dan biaya.

3.2 Stack dan Artefak Prototipe

Prototipe yang dibangun memiliki sasaran yang tegas: sebuah layanan evaluasi FHE yang dapat dipanggil aplikasi melalui HTTP, tanpa mengharuskan pengembang aplikasi menulis kode OpenFHE di sisi server. Stack dipilih agar dapat direplikasi dan dirujuk ke literatur pustaka FHE.

Mesin kriptografi memakai OpenFHE pada skema CKKS. OpenFHE dipilih karena menyediakan API terpadu untuk skema-skema FHE utama, mendukung mode pengelolaan parameter yang terpusat, dan telah dikarakterisasi secara mendalam untuk workload CKKS [11, 12]. CKKS dipilih karena sesuai untuk vektor numerik aproksimasi pada skoring linear dan sejalan dengan penggunaan skema numerik pada studi alur *secure-by-design* maupun survei PPML [3, 7]. Lapisan layanan memakai FastAPI pada Python untuk mengekspos endpoint REST, mem-parse permintaan, memanggil wrapper OpenFHE di proses backend, dan mengembalikan respons JSON. Serialisasi ciphertext memakai format biner bawaan OpenFHE yang dikodekan Base64 di dalam body JSON agar dapat melewati HTTP tanpa mengorbankan struktur ciphertext. Klien contoh ditulis dalam Python: menghasilkan kunci, mengenkripsi vektor fitur, mengirim permintaan, menerima ciphertext hasil, dan mendekripsi. Se-

bagai rujukan ekosistem Python terkait, TenSEAL menunjukkan pola operasi tensor terenkripsi di atas mesin HE, tetapi dalam penelitian ini TenSEAL tidak menjadi backend utama; yang dipakai adalah OpenFHE yang dibungkus layanan [10].

Artefak yang dihasilkan terwadahi dalam repositori prototipe HEBA. Isinya mencakup layanan backend (inisialisasi konteks CKKS, whitelist operasi, endpoint REST, pencatat metrik), klien tipis yang mematuhi standar, dokumen standar interaksi, serta skrip uji untuk baseline plaintext, alur terenkripsi, dan kasus penolakan.

Konfigurasi CKKS pada prototipe direncanakan dengan tingkat keamanan sekitar 128 bit, derajat polinomial $N = 2^{13}$ atau $N = 2^{14}$ (dipilih setelah uji kelayakan memori pada mesin pengembangan), dan kedalaman multiplikasi yang cukup untuk satu putaran perkalian elemen-wise diikuti penjumlahan, tanpa bootstrapping. Parameter final akan dicatat apa adanya pada laporan agar eksperimen dapat diulang.

3.3 Rancangan Framework Backend

Framework backend diposisikan sebagai kerangka sistem di atas OpenFHE. Aplikasi tidak memanggil API C++ OpenFHE secara langsung di server aplikasi. Aplikasi hanya berkomunikasi lewat kontrak REST yang disediakan layanan. Pola klien mengirim ciphertext, server mengevaluasi tanpa dekripsi, dan hasil dikembalikan terenkripsi mengikuti arsitektur proteksi data di *cloud* [1].

Komponen backend dirinci sebagai berikut. *Context manager* membuat dan menyimpan *crypto context* CKKS bersama pengenalan versi konteks (`context_id`). *Key material store* di sisi server hanya menampung material yang sah untuk evaluasi, misalnya *evaluation key* atau *relinearization key* yang diperlukan skema, dan tidak menyimpan secret key klien [1, 3]. *Operation registry* memetakan nama operasi ke implementasi OpenFHE yang diizinkan. *Request validator* menolak payload yang tidak cocok dengan standar sebelum ciphertext disentuh mesin FHE. *Eval engine* mengeksekusi operasi homomorfik. *Metrics logger* mencatat latensi evaluasi, ukuran payload masuk/keluar, status HTTP, dan alasan penolakan bila ada.

Endpoint yang direncanakan bersifat eksplisit. `GET /v1/context` mengembalikan `context_id`, ringkasan parameter publik, dan daftar operasi yang diizinkan. `POST /v1/eval/add` menerima dua ciphertext vektor dan mengembalikan ciphertext hasil penjumlahan. `POST /v1/eval/mul_plain` menerima

satu ciphertext dan satu vektor plaintext bobot, lalu mengembalikan hasil per-kalian elemen-wise. `POST /v1/eval/linear_score` menerima ciphertext vektor fitur $\mathbf{x}$ serta bobot $\mathbf{w}$ dan bias b dalam bentuk yang diizinkan standar, lalu mengembalikan ciphertext skalar (atau vektor slot hasil) untuk $y = \mathbf{w}^\top \mathbf{x} + b$. `GET /v1/metrics/last` mengembalikan metrik eksekusi terakhir untuk keperluan pengujian. Tidak ada endpoint dekripsi di backend.

3.4 Standar Interaksi Aplikasi dan Backend

Standar ditulis agar pengembang aplikasi memiliki kontrak yang dapat diikuti tanpa membuka detail internal OpenFHE. Setiap permintaan evaluasi wajib memuat `context_id` yang sama dengan konteks aktif backend. Body permintaan memuat ciphertext dalam Base64, dimensi vektor d , dan kode operasi. Secret key dilarang dikirim ke backend. Backend dilarang mengembalikan plaintext hasil. Operasi di luar whitelist ditolak dengan kode HTTP 400 dan pesan kesalahan generik yang tidak membocorkan isi data.

Untuk `linear_score`, standar menetapkan bahwa $\mathbf{w}$ dan b diperlakukan sebagai plaintext publik model pada prototipe ini (bukan data rahasia klien), sementara $\mathbf{x}$ adalah data sensitif yang dienkripsi klien. Pemilihan itu disengaja agar skenario dekat dengan inferensi model sederhana di server yang tidak dipercaya penuh, sejalan dengan motivasi PPML pada data numerik [7]. Dimensi d harus sama dengan konfigurasi packing yang disepakati; ketidaksesuaian dimensi ditolak validator.

Klien yang patuh standar hanya perlu: (1) mengambil deskripsi konteks publik, (2) mengenkripsi $\mathbf{x}$, (3) memanggil endpoint operasi, (4) mendekripsi hasil. Pengaturan modulus, scaling factor, dan pemanggilan primitif OpenFHE di server bukan urusan pengembang aplikasi.

3.5 Skenario Pengujian

Skenario utama adalah skoring linear terenkripsi. Dibangkitkan N_{uji} vektor fitur acak berdimensi $d = 8$ dan $d = 16$, bersama vektor bobot dan bias tetap. Untuk setiap sampel dihitung baseline plaintext $y = \mathbf{w}^\top \mathbf{x} + b$. Sampel yang sama dienkripsi di klien, dikirim ke `/v1/eval/linear_score`, didekripsi di klien, lalu dibandingkan dengan baseline. Galat relatif dan galat mutlak dicatat. Ambang keberhasilan awal merujuk pada praktik evaluasi CKKS pada komputasi numerik yang menuntut galat jauh di bawah ambang keputusan aplikasi; pada prototipe ini target operasional adalah galat relatif

di bawah 10^{-3} untuk parameter yang dipilih, dengan catatan akhir disesuaikan hasil ukur [3].

Skenario unit mencakup `add` dan `mul_plain` untuk memverifikasi bahwa registry operasi dan serialisasi ciphertext bekerja sebelum skoring linear diuji. Skenario negatif mencakup secret key yang disisipkan di body, `context_id` yang salah, dimensi yang tidak cocok, dan nama operasi di luar whitelist. Backend harus menolak keempat kasus itu.

Metrik yang dicatat per uji: waktu enkripsi klien, waktu evaluasi backend, waktu dekripsi klien, ukuran ciphertext permintaan, ukuran ciphertext respons, dan status kepatuhan. Pengukuran diulang beberapa kali pada mesin yang sama agar variasi run-to-run dapat diamati, mengikuti semangat pelaporan kelayakan pada studi FHE terapan [3, 12].

3.6 Metode Analisis

Analisis kebenaran membandingkan y hasil dekripsi terhadap baseline plaintext dan merangkum galat. Analisis kepatuhan memeriksa log penolakan pada skenario negatif serta memastikan tidak ada jalur dekripsi di server. Analisis biaya membaca latensi dan ukuran ciphertext untuk menjelaskan overhead layanan, tanpa klaim bahwa prototipe lebih cepat daripada sistem lain. Kelayakan framework dinilai dari apakah klien tipis dapat menyelesaikan skoring linear hanya dengan kontrak REST, sementara seluruh pemanggilan OpenFHE berada di backend. Temuan bottleneck dibahas dengan merujuk karakteristik CKKS/OpenFHE dan overhead FHE pada literatur sistem [12, 6, 4].

BAB IV

RENCANA PENELITIAN

4.1 Prototipe HEBA

Bukti rancangan diwujudkan sebagai prototipe **HEBA** (*Homomorphic Encryption Backend Architecture*). HEBA menempatkan evaluasi FHE pada lapisan backend, mengekspos kontrak REST yang terstandar, dan membebaskan pengembang aplikasi dari penyusunan infrastruktur kriptografi. Repositori prototipe berada pada folder `heba/` dalam folder kerja penelitian. Di dalamnya terdapat layanan evaluasi, klien tipis, penegakan standar, skrip demo, dan rangkaian uji otomatis.

Struktur perangkat lunak HEBA disusun sebagai berikut. Paket `src/heba/` memuat inti backend: profil parameter CKKS, mesin evaluasi, penegakan standar, skema payload, pencatat metrik, dan antarmuka FastAPI. Folder `client/` memuat klien Python tipis dan demo skoring linear. Folder `tests/` memuat uji mesin FHE, uji API, dan uji penolakan standar. Berkas `STANDARDS.md` memuat kontrak interaksi yang diringkas pada Lampiran A.

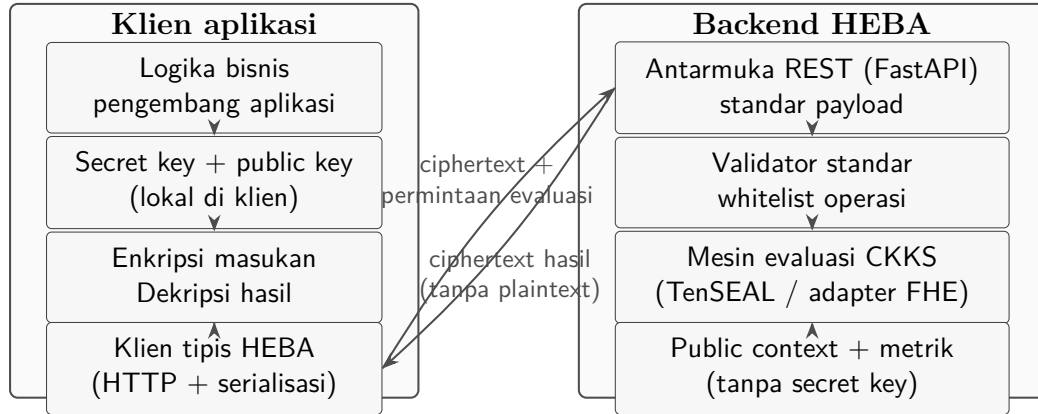
Stack yang berjalan saat ini memakai skema CKKS melalui TenSEAL (Microsoft SEAL) di balik layanan FastAPI [10, 3]. Adapter evaluasi dipisahkan agar di kemudian hari dapat diganti ke OpenFHE tanpa mengubah endpoint yang dilihat pengembang aplikasi [11, 12]. Profil parameter aktif HEBA diberi pengenal `ckks-n8192-s40-d8-16` (derajat polinomial 8192, skala 2^{40} , dimensi vektor 8 dan 16).

Arsitektur HEBA digambarkan pada Gambar IV.1. Sisi klien memegang secret key serta tugas enkripsi dan dekripsi. Sisi backend HEBA menerima public context, menegakkan standar, menjalankan evaluasi pada ciphertext, dan mengembalikan ciphertext hasil. Tidak ada jalur dekripsi di server evaluasi.

4.2 Rencana Implementasi dan Status

Implementasi disusun dalam tonggak yang dapat diperiksa, dan sebagian telah terisi oleh prototipe HEBA.

Tonggak pertama adalah mesin evaluasi CKKS yang mampu membuat konteks, menghasilkan kunci di sisi klien, mengenkripsi vektor, menjalankan



Secret key tidak pernah dikirim ke backend. Dekripsi hanya di klien.

Gambar IV.1: Arsitektur prototipe HEBA: pemisahan klien aplikasi dan backend evaluasi FHE.

penjumlahan, perkalian dengan bobot plaintext, skoring linear, serta serialisasi ciphertext. Pada HEBA, tonggak ini diwujudkan di `engine.py` dan diverifikasi oleh `tests/test_engine.py`.

Tonggak kedua adalah layanan FastAPI dengan endpoint konteks, pengikatan public context, evaluasi, dan metrik, dilengkapi validator standar. Endpoint yang disediakan meliputi `GET /v1/context`, `POST /v1/context/bind`, `POST /v1/eval/add`, `POST /v1/eval/mul_plain`, `POST /v1/eval/linear_score`, dan `GET /v1/metrics/last`. Uji API tersedia di `tests/test_api.py`.

Tonggak ketiga adalah klien tipis dan skrip demo untuk skoring linear pada $d = 8$ dan $d = 16$, dengan perbandingan terhadap baseline plaintext. Target operasional galat relatif di bawah 10^{-3} dipakai sebagai ambang awal kelayakan numerik CKKS.

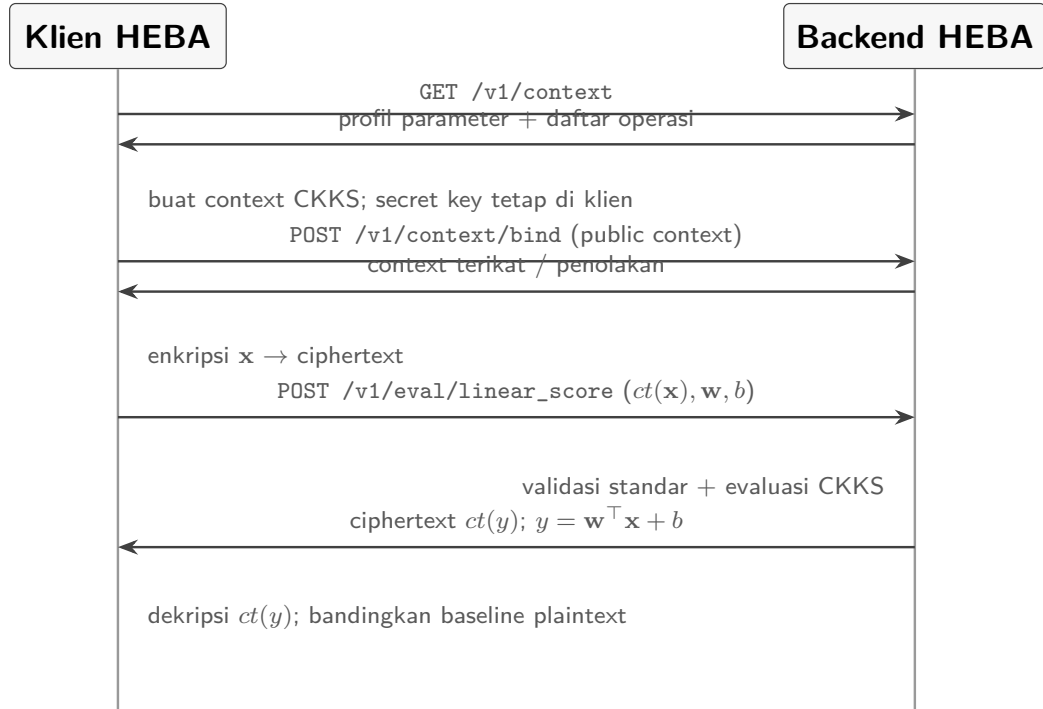
Tonggak keempat adalah dokumentasi standar interaksi. Isi kontrak ditulis di `heba/STANDARDS.md` dan diringkaskan pada Lampiran A. Pengujian penegakan standar (penolakan secret key, operasi ilegal, dan sejenisnya) ada di `tests/test_standards.py` serta skenario negatif API.

Pekerjaan lanjutan skripsi mencakup perluasan himpunan operasi, pengaitan adapter OpenFHE apabila lingkungan build sudah siap, penempatan layanan pada skenario *cloud* yang lebih realistis, serta pelaporan metrik yang lebih sistematis pada bab hasil.

4.3 Komponen dan Alur Kerja Prototipe

Komponen yang dijalankan pada HEBA mencerminkan pemisahan peran. Klien memiliki secret key, mengenkripsi masukan, dan mendekripsi keluaran. Backend hanya menyimpan public context setelah `/v1/context/bind`, menolak payload yang melanggar standar, menjalankan evaluasi pada ciphertext, dan mengembalikan ciphertext hasil. Backend tidak menyediakan endpoint dekripsi.

Alur skoring linear digambarkan pada Gambar IV.2. Klien memanggil `GET /v1/context` untuk memastikan profil parameter, membuat context CKKS lokal, lalu mengirim public context melalui `POST /v1/context/bind`. Klien mengenkripsi vektor fitur $\mathbf{x}$ dan mengirim `POST /v1/eval/linear_score` bersama bobot $\mathbf{w}$, bias b , dimensi d , dan `context_id`. Backend memvalidasi permintaan, menghitung $y = \mathbf{w}^\top \mathbf{x} + b$ secara homomorfik, dan mengembalikan ciphertext y . Klien mendekripsi y dan membandingkannya dengan baseline plaintext. Sepanjang evaluasi di server, plaintext $\mathbf{x}$ tidak direkonstruksi.



Gambar IV.2: Alur evaluasi skoring linear pada HEBA.

4.4 Rencana Hasil dan Pembahasan

Hasil yang disajikan meliputi uraian arsitektur HEBA, spesifikasi endpoint dan payload, ringkasan kontrak pada Lampiran A, tabel galat skoring

linear, tabel latensi evaluasi, serta rekap penolakan standar. Pembahasan menilai apakah pemisahan tanggung jawab tercapai: pengembang yang memakai klien tipis tidak menulis konfigurasi mesin FHE di server aplikasi, sementara backend menegakkan whitelist dan menjaga ciphertext tetap tertutup. Angka latensi dan ukuran ciphertext dibaca sebagai biaya layanan pada stack yang dipilih, lalu dikaitkan dengan temuan karakterisasi CKKS dan overhead FHE di literatur [12, 4, 6, 9].

4.5 Keterbatasan dan Arah Pengembangan

Prototipe HEBA dibatasi pada skema CKKS, himpunan operasi kecil, dan skoring linear berdimensi rendah tanpa bootstrapping. Mesin yang terpasang saat ini adalah TenSEAL; integrasi OpenFHE sebagai adapter alternatif masih menjadi pekerjaan lanjutan. Hasil tidak digeneralisasi ke jaringan dalam, operasi perbandingan, atau multi-tenant produksi. Standar masih berupa kontrak akademik REST, belum mencakup autentikasi layanan yang matang, audit forensik, atau negosiasi parameter dinamis lintas organisasi.

Arah skripsi tetap pada perancangan framework backend FHE dan standar pemakaiannya, dengan bukti jalan berupa prototipe HEBA (FastAPI, CKKS, klien tipis, dan penegakan standar). Pengembangan lanjutan dapat menambah operasi, menempatkan layanan di lingkungan *cloud*, atau menghubungkan adapter ke pustaka/akselerasi lain tanpa mengubah kontrak aplikasi.

DAFTAR PUSTAKA

- [1] A. Epishkina and V. Ermakov, “Homomorphic Encryption for Data Protection in Cloud Computing,” *Journal of Computer Virology and Hacking Techniques*, vol. 21, no. 8, 2025. [Online]. Available: <https://doi.org/10.1007/s11416-025-00552-6>
- [2] S. S. Devi and K. Jayasri, “Privacy Preserving Healthcare Based on Secure Mathematical Foundations of Fully Homomorphic Encryption (FHE),” in *2025 International Conference on Emerging Smart Computing and Informatics (ESCI)*, 2025, pp. 1–7. [Online]. Available: <https://doi.org/10.1109/ESCI63694.2025.10988128>
- [3] D. Vinko, M. Kohler, K. Milicevic, and I. Lukic, “Application of Homomorphic Encryption for a Secure-by-Design Approach to Protect the Confidentiality of Data in Proficiency Testing and Interlaboratory Comparisons,” *Telecom*, vol. 7, no. 1, p. 14, 2026. [Online]. Available: <https://www.mdpi.com/2673-4001/7/1/14/pdf>
- [4] S. Fan, X. Deng, X. Tang, W. Xu, and M. Zhang, “Understanding and Boosting Fully Homomorphic Encryption Applications on GPU,” *Cybersecurity*, vol. 9, no. 45, 2026. [Online]. Available: <https://doi.org/10.1186/s42400-025-00482-2>
- [5] Y. K. Peker and R. Raj, “Homomorphic Encryption for Confidential Statistical Computation: Feasibility and Challenges,” *Journal of Cybersecurity and Privacy*, vol. 6, no. 1, p. 4, 2025. [Online]. Available: <https://www.mdpi.com/2624-800X/6/1/4/pdf>
- [6] T. Li, Z. Wang, L. Zhao, R. Wang, Q. Niu, X. Lu, D. Meng, and R. Hou, “A Survey of Optimization Techniques for Bootstrapping Algorithms in FHE,” *Cybersecurity*, vol. 9, no. 150, 2026. [Online]. Available: <https://doi.org/10.1186/s42400-026-00571-w>
- [7] L. Wu, X. A. Wang, J. Liu, Y. Su, Z. Tu, W. Liu, H. Lei, D. Tang, Y. Cao, and J. Zhang, “Homomorphic Encryption for Machine Learning Applications with CKKS Algorithms: A Survey of Developments and Applications,” *Computers, Materials & Continua*, 2025. [Online]. Available: <https://www.techscience.com/cmc/v85n1/62800>

- [8] Z. Xu, R. Kannan, and V. Prasanna, “HERA: A Bandwidth-efficient Accelerator for Fully Homomorphic Encryption on HBM-enabled FPGA,” in *Proceedings of the 2026 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2026. [Online]. Available: <https://doi.org/10.1145/3748173.3779201>
- [9] H. Lee, A. Satriawan, and H. Lee, “Homomorphic Evaluation Cluster Architecture for Fully Homomorphic Encryption,” *IEEE Open Journal of Circuits and Systems*, vol. 6, pp. 135–146, 2025. [Online]. Available: <https://doi.org/10.1109/OJCAS.2025.3568058>
- [10] A. Benaissa, B. Retiat, B. Cebere, and A. E. Belfedhal, “TenSEAL: A library for encrypted tensor operations using homomorphic encryption,” in *ICLR 2021 Workshop on Distributed and Private Machine Learning (DPML)*, 2021. [Online]. Available: <https://arxiv.org/pdf/2104.03152.pdf>
- [11] A. Al Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, “OpenFHE: Open-source fully homomorphic encryption library,” *Cryptology ePrint Archive*, Paper 2022/915, 2022. [Online]. Available: <https://eprint.iacr.org/2022/915.pdf>
- [12] C. Brynds, P. McLeod, L. Caccamise, A. Pal, D. Saiham, S. Rahman, J. San Miguel, and D. Wu, “CryptOracle: A modular framework to characterize fully homomorphic encryption,” *arXiv preprint arXiv:2510.03565*, 2025. [Online]. Available: <https://arxiv.org/pdf/2510.03565.pdf>

BAB A

STANDAR INTERAKSI APLIKASI DAN BACKEND

Lampiran ini merangkum kontrak interaksi yang diberlakukan pada prototipe HEBA. Versi lengkap dan yang terus diperbarui berada pada berkas `heba/STANDARDS.md` di repositori penelitian. Pengembang aplikasi yang memakai backend evaluasi wajib mematuhi aturan berikut.

Pembagian peran

Klien aplikasi boleh menghasilkan kunci, mengenkripsi masukan, mengirim ciphertext, dan mendekripsi hasil. Klien tidak boleh mengirim secret key ke backend dan tidak boleh meminta dekripsi di server. Backend evaluasi boleh menyimpan public context, mengevaluasi ciphertext, dan mengembalikan ciphertext hasil. Backend tidak boleh menyimpan secret key klien dan tidak boleh mengembalikan plaintext hasil evaluasi.

Profil konteks aktif

Profil yang dipakai prototipe memiliki `context_id` `ckks-n8192-s40-d8-16`. Skema yang dipakai adalah CKKS. Derajat polinomial $N = 8192$ dan skala global 2^{40} . Dimensi vektor yang diizinkan adalah 8 dan 16. Operasi yang diizinkan adalah `add`, `mul_plain`, dan `linear_score`.

Endpoint

`GET /v1/context` mengembalikan profil parameter publik, daftar operasi, dan status pengikatan context. `POST /v1/context/bind` mengikat public context (tanpa secret key) ke backend. `POST /v1/eval/add` menjumlahkan dua ciphertext vektor. `POST /v1/eval/mul_plain` mengalikan ciphertext dengan vektor bobot plaintext. `POST /v1/eval/linear_score` menghitung $y = \mathbf{w}^\top \mathbf{x} + b$ dengan $\mathbf{x}$ terenkripsi; pada prototipe ini $\mathbf{w}$ dan b diperlakukan sebagai parameter model publik. `GET /v1/metrics/last` mengembalikan metrik evaluasi atau penolakan terakhir.

Aturan penolakan

Permintaan yang melanggar kontrak ditolak dengan HTTP 400. Alasan penolakan yang ditegakkan meliputi kehadiran field `secret_key`, ketidaksesuaian `context_id`, dimensi di luar himpunan yang diizinkan, operasi di luar whitelist, ketidaksesuaian panjang bobot, serta kegagalan pengikatan context atau evaluasi. Tidak tersedia endpoint dekripsi pada backend. Seluruh keluaran evaluasi tetap berupa ciphertext yang dikodekan Base64.

Alur pemakaian yang patuh

Pengembang aplikasi yang patuh standar menempuh urutan berikut: membaca profil lewat `GET /v1/context`; membuat context CKKS di klien dengan secret key yang tetap lokal; mengikat public context lewat `POST /v1/context/bind`; mengenkripsi masukan di klien; memanggil endpoint `/v1/eval/*`; mendekripsi hasil di klien; dan bila diperlukan membandingkan hasil dengan baseline plaintext. Target operasional galat relatif skoring linear pada profil ini adalah di bawah 10^{-3} .